

SAT Control Manager - Documentación del Proyecto (v2.2.0)

Fecha de Generación: 29 de junio de 2026

Versión del Sistema: v2.2.0 (Backend API REST + Criptografía Local en Cliente)

Nota de Seguridad: Este documento consolida el análisis de los casos de uso v1.1, el plan de implementación de seguridad de datos y el reporte del walkthrough de la versión v2.2.0 instalada y operativa localmente.

1. Resumen Ejecutivo del Proyecto

La plataforma SAT Control Manager centraliza y automatiza el monitoreo de vigencias de las firmas electrónicas avanzadas (FIEL/e.firma) para despachos contables y corporativos, eliminando omisiones operativas y multas por vencimientos sorpresa mediante un semáforo interactivo y notificaciones automatizadas.

2. Análisis Comparativo de Casos de Uso (v1.0 vs v1.1)

El análisis detallado del documento de casos de uso y comparativas identificó fallos de seguridad críticos en la versión anterior que fueron completamente resueltos en esta versión:

Caso de Uso (CU)	Vulnerabilidad v1.0	Solución de Seguridad v1.1 / v2.2.0
CU-01: Registro de Firma	Contraseña de clave privada transmitida en texto plano al servidor.	Cifrado Simétrico Local: La contraseña nunca sale del navegador. Se cifra en cliente mediante AES-GCM-256.
CU-02: Motor Semafórico	Desfase de días por diferencia de husos horarios.	Cálculo estricto y normalizado en el servidor a las **00:00 UTC** .

Caso de Uso (CU)	Vulnerabilidad v1.0	Solución de Seguridad v1.1 / v2.2.0
CU-04: Consulta de Claves	Acceso ilimitado a las contraseñas sin rastreo formal ni bitácoras inalterables.	Límite estricto de **10 consultas diarias** por operador, validación **2FA TOTP** y logs ledger-chain SHA-256.
CU-05: Alta de Usuarios	Contraseñas iniciales asignadas por el administrador de forma insegura.	Generación de token de onboarding de un solo uso para que el operador la definina privadamente.

3. Arquitectura del Backend e Infraestructura

El sistema se construyó con la siguiente pila tecnológica y de seguridad:

- **Express Server (Node.js):** API REST modular organizada en controladores, middlewares y utilidades.
- **Base de Datos Relacional:** Implementada en SQLite a través de `better-sqlite3`, optimizando rendimiento y permitiendo portabilidad absoluta.
- **Inmutabilidad Ledger-Chain:** Cada entrada de bitácora calcula su firma digital usando el hash del bloque anterior. Cualquier borrado o actualización corrompe la cadena y es reportado instantáneamente.

Esquema de Tablas (BD):

- `usuarios` : Roles RBAC (admin, supervisor, operador), secretos de 2FA TOTP y tokens de onboarding.
- `contribuyentes` : Expedientes con RFC, vigencia y la llave privada cifrada localmente.
- `historial_renovaciones` : Archivo histórico de certificados de contribuyentes archivados al renovar (CU-01b).
- `consultas_contrasena_log` : Historial de consultas diarias para hacer valer la cuota límite.
- `bitacora_logs` : Ledger de auditoría criptográfica.

4. Walkthrough de Cambios y Verificación

Los componentes desarrollados y validados son:

1. **Servidor Backend:** Escuchando en el puerto `3001` . Conexión a base de datos validada.
2. **Rutas API y Permisos:** Validado el correcto funcionamiento del login y generación de JWT mediante scripts de PowerShell.
3. **Criptografía Local:** Integración en `index.html` de la API Web Crypto del navegador para que el cifrado AES-GCM ocurra localmente antes de hacer el POST.
4. **Bitácora ledger y Verificación de Cadena:** Endpoint ``/api/bitacora/integridad`` que audita toda la base de datos y certifica si es inmutable.

5. Instrucciones para la Puesta en Marcha Local

Para levantar y probar el sistema localmente con la base de datos relacional activa:

```
# 1. Instalar dependencias
npm install

# 2. Inicializar la Base de Datos SQLite y Semillas
npm run init-db

# 3. Arrancar el Servidor Backend Express
npm run dev
```

Una vez arrancado, abre `index.html` y autentícate con la cuenta semilla:

- **Correo:** `admin@fiel.mx`
- **Contraseña:** `Admin1234.`